Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

MemOpt: Better Memory Use with Access Time and Memory-Level Parallelism

Team Name: *Team #*	*TierX*
Team member 1 (Team Lead)	*Bhargava, Garima – 240121232* *garimabhargava4@gmail.com* 
Team member 2	*Bhatt, Nandini – 24022559* *nandinibhatt006@gmail.com* 
Team member 3	*Tomar, Shaili – 240221680* *shailitomar0112@gmail.com* 
Team member 4	*Bisht, Saumya – 240221898* *saumyabisht179@gmail.com* 

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

MemOpt is a project about computer memory. It tries to make a computer work better by keeping useful data in fast memory. The old method checks how many times data is used. But data used many times may not always need fast memory the most.
MemOpt uses a simple score called AOL to rank data. This score helps us decide which data may need fast memory. We will compare it with the old hotness method.
We use C++ to write the main code and run tests. We plan to use PostgreSQL to save work details and test results. We use separate folders to show fast and slow memory.
We will test both methods with similar work and the same amount of fast memory. The results will help us see if using access time is better than only counting how often data is used.

Updated Project Approach and Architecture

We are making a simple computer model of MemOpt. It will compare the old hotness method with a method that uses the AOL score.
System Design and Architecture: The system has six main parts. It runs a test. It collects data use details. It works out the AOL score. It ranks the data. It puts data in fast or slow memory folders. It then checks the results. We will use real values when we can. If we cannot get real values we will use sample values and mark them clearly. The hotness method ranks data by how many times it is used. MemOpt ranks data by its AOL score. Both methods will use the same memory limits so the test is fair.
Technology stack/languages/tools: We will write the main code in C++. When the database link is ready PostgreSQL will save test details and results. We will use Visual Studio Code to write the code. C++ tools will help us store data do maths and read or write files. We may use Linux perf or hardware counters if our computer supports them.
Communication and data flow: All parts of the system will share information. We plan to save test details and results in PostgreSQL with SQL and its C++ library. The model should not need a network link between computers or programs.
System workflow: The system will run a test collect data use details work out the AOL score rank the data put it in fast or slow memory folders check the results and save them in PostgreSQL.
The project does not need real CXL memory. The tools we can use to check memory will depend on our computer and operating system.

Tasks Completed

Task Completed	Team Member
Found the problem and studied the old way to place data in memory	Garima
Set the project goals and planned the AOL method	Nandini
Planned the system and its main parts Chose C++ and PostgreSQL	Shaili
Planned the hotness method for comparison	Saumya
Planned folders to show fast and slow memory	Garima and Nandini
	Shaili and Saumya

Challenges/Roadblocks

We have some problems to solve. It may be hard to get real data about how often memory is used how long access takes and MLP. Some computers do not have the tools we need. We will use the tools we have and mark sample values clearly.
We also need to work out the AOL score the right way. We will explain our formula and test it with sample values.
Both methods must use the same test work and the same amount of fast memory. This will make the test fair.
We may not have the hardware to test real CXL memory or move real memory pages. So we will use folders to show fast and slow memory.
We also need to link the C++ program to PostgreSQL and save the results. We will do this one step at a time.
We will test each part and then test the full system.

Tasks Pending

Task Pending	Team Member (to complete the task)
Make the hotness and AOL methods	Garima
Make the parts that check memory use and work out AOL	Nandini
Make the parts that place and move data	Shaili
Link C++ with PostgreSQL	Saumya
Run tests and check the results	Garima and Nandini
Finish the tests and project papers	All members

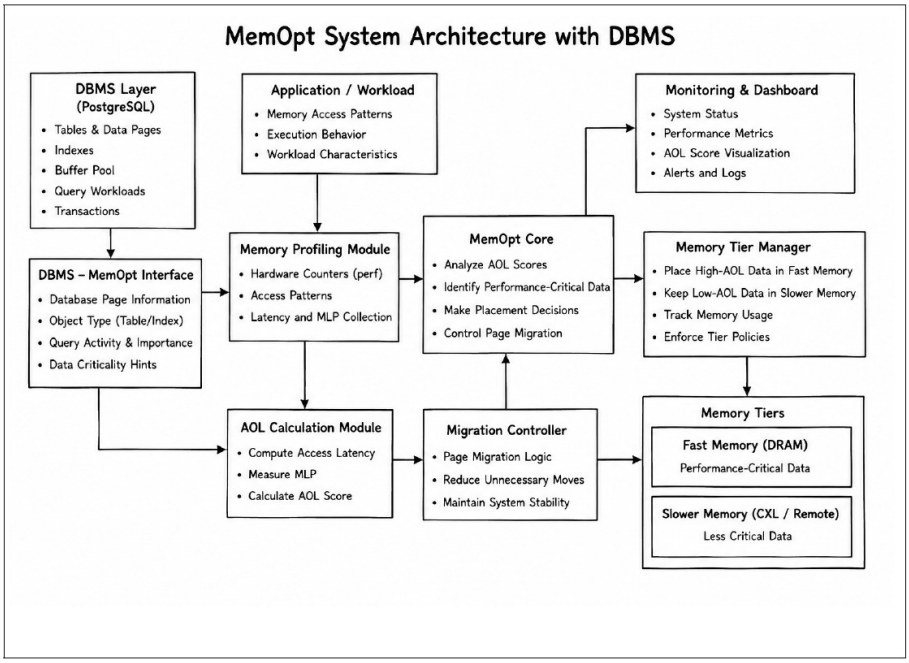
Project Outcome/Deliverables

Our goal is to make a working model of MemOpt. We will compare the old hotness method with the AOL method.
The main things we plan to make are:
1. C++ Program: Runs the code that places data.
2. Hotness Method: Places data by how many times it is used.
3. AOL Method: Ranks data with the AOL score.
4. Memory Manager: Puts data in folders that show fast and slow memory.
5. PostgreSQL Database: Saves test details and results.
6. Tests: Compare both methods using the same test work.

This project will help us learn how these methods change system speed. It also uses ideas from Operating Systems and DBMS. Our model uses folders to show memory types. We will not say it uses real CXL memory unless we build and test it.

Progress Overview

We have found the problem and listed what the project needs. We have chosen the main tools and planned the system. We have planned the hotness method the AOL method the memory folders and PostgreSQL.
The next step is to write the code. We need to make both methods link C++ to PostgreSQL and test the AOL score and memory manager.
We will run tests when the main parts are ready. If we cannot get real memory data we will use test data and mark it clearly.
Next we will test each part and join them into one system. We will compare both methods using the same test work and memory limits.



Codebase Information

Repository: https://github.com/garima1-tech/MemOpt
Main Branch: `main`
Programming Language: C++
Database: PostgreSQL
Development Environment: Visual Studio Code
Important Modules:
AOLPolicy.cpp and AOLPolicy.h – Choose where data goes using AOL
TierManager.cpp and TierManager.h – Place data in memory
schema.sql – Makes the database tables
workload.sql – SQL tests
architecture.md – System plan
setup.md – Setup steps
Important Commits:
Add AOL policy logic – Added the AOL files.
Add TierManager – Added the files that place data.
Create workload.sql – Added test queries.
Add schema.sql – Added the database table setup.
Add files via upload – Added project files and papers.
Repository status: The repository has 11 commits and one branch called `main`.

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
C++ compilation	Pass	Check that the code builds.
PostgreSQL connection	Pass	Check that the program links to the database.
Database operations	Pending	Check that test results can be saved and read.
Hotness based placement	Pending	Check where data is placed using sample counts.
AOL calculation	Pass	Check the score using known values.

Deliverables Progress

Deliverable	Current status
Project problem and needs	Completed
System plan	Completed
C++ project files	In progress
Hotness based placement method	In progress
AOL based placement method	Pending
Memory manager	Pending
PostgreSQL link	Pending